Data Handling: Import, Cleaning and Visualisation

Solutions to Exercise/Workshop 3: Importing data from text files, XML and HTML

Dr. Aurélien Sallin (Lecturer) Ana Armendariz (Tutor)

15 October 2026

Table of contents

Exercise 1: importing a CSV in two ways 1

Exercise 2: column types 3

Exercise 3: select what you need and write it back 5

Exercise 4: the same data in XML 6

Exercise 5: a file that is not comma separated 8

Exercise 6: what the browser shows and what the server sent 10

Additional exercises for training 11

Exercise 7: scraping a table from a web page 11

The questions are repeated here in *italics*, so that this document can be read on its own. The data files are in `materialsExercises/03_programming_data/data`.

Exercise 1: importing a CSV in two ways

1. *Import the file twice, once with base R and once with* ***`readr`****.*

   Base R has `read.csv()`, the `readr` package has `read_csv()`:

```
air_quality_base <- read.csv("data/Air_Quality.csv")

library(readr)
```

```
Warning: package 'readr' was built under R version 4.5.3
```

```
air_quality <- read_csv("data/Air_Quality.csv", show_col_types = FALSE)
```

2. *Inspect what you imported with* ***`head()`****,* ***`tail()`****,* ***`dim()`****, and* ***`glimpse()`*** *from* ***`dplyr`****. How many rows and columns are there?*

   16,122 rows and 12 columns:

```r
dim(air_quality)
```

```
[1] 16122    12
```

```r
head(air_quality[, c("Name", "Geo Place Name", "Time Period", "Data Value")])
```

```
# A tibble: 6 x 4
  Name                `Geo Place Name`                     `Time Period`  `Data Value`
  <chr>               <chr>                                <chr>                 <dbl>
1 Ozone (O3)          Coney Island (CD13)                  Summer 2013            34.6
2 Ozone (O3)          Coney Island (CD13)                  Summer 2014            33.2
3 Ozone (O3)          Bronx                                Summer 2013            31.2
4 Ozone (O3)          Bronx                                Summer 2014            31.2
5 Sulfur Dioxide (SO2) Morris Park and Bronxdale (CD11)    Winter 2008-09          5.89
6 Sulfur Dioxide (SO2) Williamsbridge and Baychester (CD12) Winter 2008-09         5.75
```

```r
library(dplyr)
```

```
Warning: package 'dplyr' was built under R version 4.5.3
```

```r
glimpse(air_quality)
```

```
Rows: 16,122
Columns: 12
$ `Unique ID`      <dbl> 216498, 216499, 219969, 219970, 164876, 164877, 219971, 219972, 164878, 1~
$ `Indicator ID`   <dbl> 386, 386, 386, 386, 383, 383, 386, 386, 383, 383, 383, 383, 383, 383, 383~
$ Name             <chr> "Ozone (O3)", "Ozone (O3)", "Ozone (O3)", "Ozone (O3)", "Sulfur Dioxide (~
$ Measure          <chr> "Mean", "Mean", "Mean", "Mean", "Mean", "Mean", "Mean", "Mean", "Mean", "~
$ `Measure Info`   <chr> "ppb", "ppb", "ppb", "ppb", "ppb", "ppb", "ppb", "ppb", "ppb", "ppb", "pp~
$ `Geo Type Name`  <chr> "CD", "CD", "Borough", "Borough", "CD", "CD", "Borough", "Borough", "CD",~
$ `Geo Join ID`    <dbl> 313, 313, 1, 1, 211, 212, 2, 2, 301, 302, 303, 304, 305, 306, 307, 308, 2~
$ `Geo Place Name` <chr> "Coney Island (CD13)", "Coney Island (CD13)", "Bronx", "Bronx", "Morris P~
$ `Time Period`    <chr> "Summer 2013", "Summer 2014", "Summer 2013", "Summer 2014", "Winter 2008-~
$ Start_Date       <chr> "06/01/2013", "06/01/2014", "06/01/2013", "06/01/2014", "12/01/2008", "12~
$ `Data Value`     <dbl> 34.64, 33.22, 31.25, 31.15, 5.89, 5.75, 26.27, 33.83, 4.33, 4.41, 4.73, 4~
$ Message          <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
```

The same indicator appears in many rows, once per area and per time period.

3. *Compare* ***`names(air_quality_base)`*** *with* ***`names(air_quality)`****. The file has a column called* ***`Unique ID`****, with a space in the name. What did each function do with that name, and how do you refer to the column afterwards?*

```r
names(air_quality_base)
```

```
 [1] "Unique.ID"      "Indicator.ID"   "Name"           "Measure"        "Measure.Info"
 [6] "Geo.Type.Name"  "Geo.Join.ID"    "Geo.Place.Name" "Time.Period"    "Start_Date"
[11] "Data.Value"     "Message"
```

```
names(air_quality)
```

```
 [1] "Unique ID"      "Indicator ID"   "Name"           "Measure"        "Measure Info"
 [6] "Geo Type Name"  "Geo Join ID"    "Geo Place Name" "Time Period"    "Start_Date"
[11] "Data Value"     "Message"
```

`read.csv()` repairs the names and turns spaces into dots: `Unique ID` becomes `Unique.ID`, so you write `air_quality_base$Unique.ID`. `read_csv()` keeps the original names, so a name with a space needs backticks:

```
head(air_quality$`Unique ID`, 3)
```

```
[1] 216498 216499 219969
```

Neither is wrong, but you need to know which one you are using.

4. *Compare `class(air_quality_base)` with `class(air_quality)`. What is the difference between the two objects?*

```
class(air_quality_base)
```

```
[1] "data.frame"
```

```
class(air_quality)
```

```
[1] "spec_tbl_df" "tbl_df"      "tbl"         "data.frame"
```

`read.csv()` returns a `data.frame`, `read_csv()` returns a `tibble`: it prints ten rows instead of all of them and shows the type of every column under its name. A tibble is still a data frame, which you can see from `data.frame` in its class.

5. *`read_csv()` printed a message about column specifications, `read.csv()` printed nothing. Which behaviour do you prefer when you open a file for the first time, and why?*

   The message, for a file you do not know: a column of numbers that was read as text shows up immediately instead of three steps later.

Exercise 2: column types

1. *Print the full specification with `spec(air_quality)`. Which columns came out as `col_double()` and which as `col_character()`?*

```
spec(air_quality)
```

```
cols(
  `Unique ID` = col_double(),
  `Indicator ID` = col_double(),
  Name = col_character(),
  Measure = col_character(),
  `Measure Info` = col_character(),
  `Geo Type Name` = col_character(),
  `Geo Join ID` = col_double(),
  `Geo Place Name` = col_character(),
  `Time Period` = col_character(),
  Start_Date = col_character(),
  `Data Value` = col_double(),
  Message = col_logical()
)
```

Four columns were guessed as `col_double()`, the two IDs, the join ID and `Data Value`. The rest are `col_character()`.

2. *`Start_Date` contains dates such as `06/01/2013`, but it was not read as a date. Why can the parser not recognise it, and what would the file have to look like for `readr` to guess a date by itself?*

```
head(air_quality$Start_Date, 3)
```

```
[1] "06/01/2013" "06/01/2014" "06/01/2013"
```

`readr` guesses dates only in the ISO format, `YYYY-MM-DD`. `06/01/2013` is ambiguous: 1 June 2013 in the American convention, 6 January 2013 in the European one. Rather than pick one, `readr` leaves the column as text. With `2013-06-01` in the file it would have been parsed as a date without help.

3. *State the type yourself with `col_types = cols(Start_Date = col_date(format = "%m/%d/%Y"))`. Check the class of the column, then run `range()` on it. Which period does the data cover? Run `range()` on the text version too, and explain the difference.*

```
air_quality_dates <- read_csv(
  "data/Air_Quality.csv",
  col_types = cols(Start_Date = col_date(format = "%m/%d/%Y"))
)

class(air_quality_dates$Start_Date)
```

```
[1] "Date"
```

```
range(air_quality_dates$Start_Date)
```

```
[1] "2005-01-01" "2020-06-01"
```

The data covers January 2005 to June 2020. On the text version the answer is different:

```
range(air_quality$Start_Date)
```

```
[1] "01/01/2005" "12/31/2015"
```

Strings are sorted alphabetically, so the smallest is `"01/01/2005"` and the largest `"12/31/2015"`: a string starting with `12/` sorts after one starting with `06/`, whatever the year follows. Neither value is the first or the last date in the data. The answer looks plausible and is wrong.

4. *Why is it better to fix a type at import than to convert the column afterwards?*

   Every later step inherits the type, and a date stored as text cannot be sorted, subtracted or plotted on a time axis. Stating the type also records, in the script, what you expect the file to contain: the day the provider changes the format, the import fails instead of quietly producing `NA`.

Exercise 3: select what you need and write it back

1. *`Name` holds the indicator that was measured. List the distinct values with `unique(air_quality$Name)`. How many are there?*

   Nineteen, from ozone to traffic density:

```
length(unique(air_quality$Name))
```

```
[1] 19
```

```
unique(air_quality$Name)
```

```
 [1] "Ozone (O3)"
 [2] "Sulfur Dioxide (SO2)"
 [3] "PM2.5-Attributable Deaths"
 [4] "Boiler Emissions- Total SO2 Emissions"
 [5] "Boiler Emissions- Total PM2.5 Emissions"
 [6] "Boiler Emissions- Total NOx Emissions"
 [7] "Air Toxics Concentrations- Average Benzene Concentrations"
 [8] "Air Toxics Concentrations- Average Formaldehyde Concentrations"
 [9] "PM2.5-Attributable Asthma Emergency Department Visits"
[10] "PM2.5-Attributable Respiratory Hospitalizations (Adults 20 Yrs and Older)"
[11] "PM2.5-Attributable Cardiovascular Hospitalizations (Adults 40 Yrs and Older)"
[12] "Traffic Density- Annual Vehicle Miles Traveled"
[13] "O3-Attributable Cardiac and Respiratory Deaths"
[14] "O3-Attributable Asthma Emergency Department Visits"
[15] "O3-Attributable Asthma Hospitalizations"
[16] "Traffic Density- Annual Vehicle Miles Traveled for Cars"
[17] "Traffic Density- Annual Vehicle Miles Traveled for Trucks"
[18] "Nitrogen Dioxide (NO2)"
[19] "Fine Particulate Matter (PM2.5)"
```

2. *Keep the rows measuring fine particulate matter in the winter of 2008-09 in Lower Manhattan, with `subset()` and `&`. How many rows are left, and what is the measured value?*

```
pm25_winter <- subset(air_quality,
                      Name == "Fine Particulate Matter (PM2.5)" &
                      `Time Period` == "Winter 2008-09" &
                      `Geo Place Name` == "Lower Manhattan")

dim(pm25_winter)
```

```
[1]  1 12
```

```
pm25_winter$`Data Value`
```

```
[1] 15.79
```

One row: in winter 2008-09, fine particulate matter in Lower Manhattan was measured at 15.79 mcg/m³. Use `&`, not `&&`: `&&` looks only at the first element and returns one `TRUE` or `FALSE`, which keeps either everything or nothing.

3. *Save the result as `airquality_manhattan.csv` with `write_csv()`, then open the file in a text editor.*

```
write_csv(pm25_winter, "data/airquality_manhattan.csv")
```

The file holds a header line with the twelve column names and one line of values. `write_csv()` writes UTF-8 and, unlike `write.csv()`, adds no column of row numbers. For the number alone, use `writeLines(as.character(pm25_winter$"Data Value"), "data/airquality_manhattan.txt")`.

4. *Now keep every measurement of fine particulate matter in Lower Manhattan, for all time periods, and save it the same way. If you read `Start_Date` as a date in Exercise 2, sort the series by date first.*

   Dropping the condition on the time period leaves the whole series for that neighbourhood:

```
pm25_all <- subset(air_quality,
                   Name == "Fine Particulate Matter (PM2.5)" &
                   `Geo Place Name` == "Lower Manhattan")

dim(pm25_all)
```

```
[1] 36 12
```

```
pm25_sorted <- pm25_all[order(as.Date(pm25_all$Start_Date, format = "%m/%d/%Y")), ]
head(pm25_sorted[, c("Time Period", "Start_Date", "Data Value")], 4)
```

```
# A tibble: 4 x 3
  `Time Period`       Start_Date `Data Value`
  <chr>               <chr>             <dbl>
1 Winter 2008-09      12/01/2008         15.8
2 Annual Average 2009 12/01/2008         13.1
3 Summer 2009         06/01/2009         13.2
4 Winter 2009-10      12/01/2009         12.4
```

```
write_csv(pm25_sorted, "data/airquality_manhattan_all.csv")
```

Sorting the text column instead puts December before February, for the same reason `range()` misbehaved in Exercise 2.

Exercise 4: the same data in XML

1. *Load the file with `xml2`. What kind of object did you get? Is it a data frame?*

```
library(xml2)
```

```
Warning: package 'xml2' was built under R version 4.5.3
```

```
doc <- read_xml("data/Air_Quality.xml")
class(doc)
```

```
[1] "xml_document" "xml_node"
```

```
xml_name(doc)
```

```
[1] "row"
```

Not a data frame: an object representing the XML tree, which holds everything we need but cannot be used as a table. The root element is called `row`.

2. *Collect the measurements with `xml_find_all(doc, "//row/row")` and count them. Compare the number with the number of rows in the CSV.*

   Each measurement is a `row` element inside a root that is also called `row`, which is why the XPath expression has `row` twice:

```
rows <- xml_find_all(doc, "//row/row")
length(rows)
```

```
[1] 16122
```

16,122, the same as the CSV. The two files hold the same data in two formats.

3. *Inspect the first measurement: the names of its children, and their contents.*

   The children are the variables, their text is the values:

```
xml_name(xml_children(rows[[1]]))
```

```
 [1] "unique_id"      "indicator_id"   "name"           "measure"        "measure_info"
 [6] "geo_type_name"  "geo_join_id"    "geo_place_name" "time_period"    "start_date"
[11] "data_value"
```

```
xml_text(xml_children(rows[[1]]))
```

```
 [1] "216498"              "386"                 "Ozone (O3)"          "Mean"
 [5] "ppb"                 "CD"                  "313"                 "Coney Island (CD13)"
 [9] "Summer 2013"         "2013-06-01T00:00:00" "34.64"
```

4. *Extract one variable for all measurements with an XPath expression, for example every `data_value`. What type is the result, and what do you have to do before you can compute the mean?*

```
data_value <- xml_text(xml_find_all(doc, "//row/row/data_value"))
length(data_value)
```

```
[1] 16122
```

```
head(data_value, 5)
```

```
[1] "34.64" "33.22" "31.25" "31.15" "5.89"
```

```
typeof(data_value)
```

```
[1] "character"
```

Text. XML has no column types, so everything arrives as character and you convert before computing:

```
mean(as.numeric(data_value))
```

```
[1] 19.12916
```

5. *Compare* `start_date` *here with* `Start_Date` *in the CSV, and the size of the two files with* `file.size()`*. What does XML give you that a CSV cannot, and what do you pay for it?*

```
head(xml_text(xml_find_all(doc, "//row/row/start_date")), 2)
```

```
[1] "2013-06-01T00:00:00" "2014-06-01T00:00:00"
```

```
round(file.size("data/Air_Quality.xml") / 1024^2, 1)    # MB
```

```
[1] 8.7
```

```
round(file.size("data/Air_Quality.csv") / 1024^2, 1)    # MB
```

```
[1] 1.9
```

The XML carries ISO timestamps, `2013-06-01T00:00:00`, which `readr` parses without a format string; the CSV has the ambiguous `06/01/2013`.

XML stores structure: elements can be nested, repeated and labelled, every value carries its variable name, and a document can describe its own rules. A CSV holds a flat table and nothing else. You pay in size and speed, 8.7 MB against 1.9 MB for the same data, because every value is wrapped in its tags, and you need XPath to get anything out. For a flat table, the CSV is easier.

Exercise 5: a file that is not comma separated

1. *Open* `data/temp_anomaly.txt` *in a text editor, or print its first lines from R. How many lines of metadata are there, and on which line does the first observation start?*

```
readLines("data/temp_anomaly.txt", n = 12)
```

```
 [1] "             VARIABLE : Surface temperature anomaly (K)"
 [2] "             DATA SET : GISTEMP Surface Temperature Analysis"
 [3] "             FILENAME : gistemp1200_GHCNv4_ERSSTv5.nc"
 [4] "             FILEPATH : /usr/local/fer_dsets/data/GISTEMP/"
 [5] "             BAD FLAG : -1.E+34       "
 [6] "             SUBSET   : 1713 points (TIME)"
 [7] "             LONGITUDE: 9W(-9)"
 [8] "             LATITUDE : 47N"
 [9] "         \t9W(-9)  "
[10] "JAN-1880\t0.33"
[11] "FEB-1880\t0.3"
[12] "MAR-1880\t0.47"
```

Nine lines of metadata: variable, data set, file name, file path, bad flag, subset, longitude, latitude, and a line repeating the coordinates. The first observation, `JAN-1880`, is on line 10.

The `\t` between the month and the value is the tab character, which is what `read_tsv()` uses as separator in question 2.

2. *Read the file with `read_tsv()`, `skip = 9` and `col_names = c("month", "temp_anomaly")`.*

```
anomalies <- read_tsv("data/temp_anomaly.txt",
                      skip = 9,
                      col_names = c("month", "temp_anomaly"),
                      show_col_types = FALSE)
head(anomalies)
```

```
# A tibble: 6 x 2
  month    temp_anomaly
  <chr>           <dbl>
1 JAN-1880         0.33
2 FEB-1880         0.3
3 MAR-1880         0.47
4 APR-1880         0.15
5 MAY-1880        -0.36
6 JUN-1880        -0.43
```

3. *What is the purpose of the `skip` argument, and why is it set to 9? And why does `col_names` have to be given here?*

   `skip = 9` drops the metadata before parsing starts, so it stays out of the table. `col_names` is needed because the first line after the metadata is already an observation: there is no header row to take the names from. Without it, `JAN-1880` and `0.33` would have become the column names and the series would be one observation short.

4. *Why do we use `read_tsv()` and not `read_csv()`? Try it with `read_csv()`: how many columns do you get, and what is inside them?*

   The separator is a tab, not a comma:

```
anomalies_wrong <- read_csv("data/temp_anomaly.txt", skip = 9,
                            col_names = "everything", show_col_types = FALSE)
head(anomalies_wrong, 3)
```

```
# A tibble: 3 x 1
  everything
  <chr>
1 "JAN-1880\t0.33"
2 "FEB-1880\t0.3"
3 "MAR-1880\t0.47"
```

Finding no comma, `read_csv()` reads each line as a single field: one column, with the tab character `\t` still inside the values, and the anomaly as text instead of a number. `read_tsv()` is `read_csv()` with the tab as delimiter, and `read_delim()` takes any delimiter, which is what the `:` separated file in the Lecture 4 homework needs.

5. *How many observations does the series contain, and what is the mean anomaly?*

```
nrow(anomalies)
```

```
[1] 1713
```

```
mean(anomalies$temp_anomaly)
```

```
[1] 0.09798015
```

```
summary(anomalies$temp_anomaly)
```

```
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
-1.78000 -0.28000  0.07000  0.09798  0.48000  1.79000
```

1713 monthly observations from January 1880 on, mean anomaly about 0.1 K, minimum −1.78, maximum 1.79. The largest values are the recent ones.

Exercise 6: what the browser shows and what the server sent

1. and 2. *Open the page, inspect the table, and say which tags hold the years and which hold the party names.*

   The table is a `<table class="wikitable">`, and each Congress is one `<tr>`. Inside a row, the year and the party names each sit in their own `<td>`; the first row uses `<th>` for the column names. That regularity is what `html_table()` exploits in Exercise 7: rows become rows, cells become columns.

2. *Replace the `<h1>` title with one of your own. What happens on the screen?*

   The title changes immediately. The browser holds a copy of the document in memory, the DOM, and renders that copy; your edit changes the copy.

3. *Read the same page with R and print its first heading. Does your title appear? Why not?*

```
library(rvest)

doc <- read_html("https://en.wikipedia.org/wiki/Divided_government_in_the_United_States")
html_text(html_element(doc, "h1"))
```

```
[1] "Divided government in the United States"
```

The real title comes back, not yours. `read_html()` asks Wikipedia's server for the page and parses the HTML the server sends; it never sees your browser. Your edit existed in one tab, on one computer.

5. *Reload the page. Where did your edit go, and who else saw it?*

   It is gone, and nobody saw it. Reloading throws the edited copy away and renders the document the server sends again.

6. *What does this tell you about the difference between what a website shows and what the data behind it is?*

   A website is a rendering of text that a server sent you, and the rendering can be changed at will. Two consequences: the data behind a page is accessible, which is what makes Exercise 7 possible, and a screenshot of a website is not evidence of anything.

Additional exercises for training

Exercise 7: scraping a table from a web page

Build a CSV file with data on divided government in US politics out of the Wikipedia page on divided government in the United States: fetch the page with `read_html()`, pick the table with `Year`, `President`, `Senate` and `House`, drop the footnote row at the end, turn every range of years into one row per year, remove Wikipedia's footnote markers, and write the result to `data/divided_gov.csv`.

The code needs an internet connection, so it is shown here rather than executed.

```
library(rvest)
library(dplyr)
library(stringr)
library(readr)

SOURCE_PATH <- "https://en.wikipedia.org/wiki/Divided_government_in_the_United_States"
OUTPUT_PATH <- "data/divided_gov.csv"

# 1. fetch the whole HTML document
doc <- read_html(SOURCE_PATH)

# 2. extract the tables and pick the right one
tables <- html_table(doc, fill = TRUE)
length(tables)
tab <- tables[[2]]        # the one with Year, President, Senate, House

# 3. check the last row and drop it if it is a footnote
tail(tab, 2)
tab <- tab[-nrow(tab), ]

# 4. one row per year instead of one row per Congress
all_years <- list()

for (i in seq_len(nrow(tab))) {
  row <- tab[i, ]
  pieces <- unlist(strsplit(row$Year, split = "[\\-\\-]", perl = TRUE))
  begin <- as.numeric(pieces[1])
  end <- as.numeric(pieces[2])

  all_years[[i]] <- data.frame(year = begin:(end - 1),
                               president = row$President,
                               senate = row$Senate,
                               house = row$House)
}
```

```
divided_gov <- bind_rows(all_years)

# 5. remove Wikipedia's footnote markers
divided_gov$senate <- str_remove_all(divided_gov$senate, "\\[[^\\]]*\\]")
divided_gov$house <- str_remove_all(divided_gov$house, "\\[[^\\]]*\\]")

# 6. inspect and write to disk
head(divided_gov)
write_csv(divided_gov, OUTPUT_PATH)
```

The result is also in the `data` folder:

```
divided_gov <- readr::read_csv("data/divided_gov.csv", show_col_types = FALSE)
dim(divided_gov)
```

```
[1] 164   4
```

```
head(divided_gov, 4)
```

```
# A tibble: 4 x 4
   year president senate house
  <dbl> <chr>     <chr>  <chr>
1  1861 Lincoln   R      R
2  1862 Lincoln   R      R
3  1863 Lincoln   R      R
4  1864 Lincoln   R      R
```

Three remarks on the method.

`html_table()` returns a list, because a page usually has more than one table, so you have to look at the candidates and choose. `fill = TRUE` pads rows with fewer cells than the header, which is common in hand-written Wikipedia tables.

The split pattern `"[\\-\\-]"` matches two characters, the ASCII hyphen `-` and the typographic en dash `–`. Wikipedia uses both, and a pattern with only the hyphen fails on part of the rows without saying so.

`begin:(end - 1)` and not `begin:end`: a Congress from 1861 to 1863 covers 1861 and 1862, and 1863 belongs to the next one. Off-by-one errors are the usual mistake when reshaping periods into years, and the way to catch them is to count a couple of rows by hand.